

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 4

## Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |
| Name & Reg. No.     | Sadhumitha S.                                 | 24011101091 |

### 1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

### 2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

### 3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

## 4. Evolution of CNN Architectures

| Model     | Year | Major Contribution                                   |
|-----------|------|------------------------------------------------------|
| LeNet-5   | 1998 | First practical convolutional neural network         |
| AlexNet   | 2012 | ReLU activation, Dropout and GPU training            |
| VGG16     | 2014 | Uniform $3 \times 3$ convolution filters             |
| GoogleNet | 2014 | Inception modules for multi-scale feature extraction |
| ResNet    | 2015 | Residual learning using skip connections             |

## 5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

## 6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

## 7. VGG16

VGG16 demonstrated that using multiple small  $3 \times 3$  convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

## 8. Comparison of Architectures

| Architecture | Depth | Parameters | Main Contribution         |
|--------------|-------|------------|---------------------------|
| LeNet-5      | 7     | 60K        | First CNN                 |
| AlexNet      | 8     | 61M        | ReLU + Dropout            |
| VGG16        | 16    | 138M       | Deep $3 \times 3$ filters |
| GoogleNet    | 22    | 6.8M       | Inception Modules         |
| ResNet50     | 50    | 25.6M      | Residual Learning         |

## 9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies  $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$  convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of  $1 \times 1$  convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

## 10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$  = Residual Mapping
- $x$  = Identity Mapping

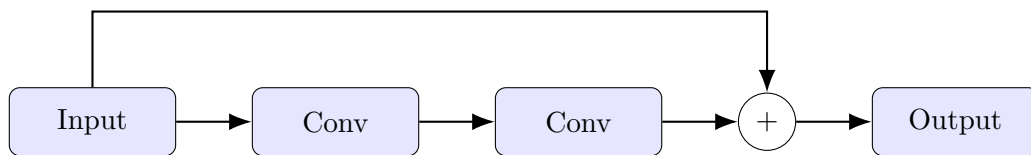


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

## 11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

## 12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

## 13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

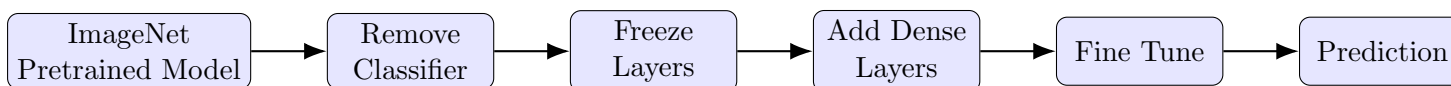


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

## 14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size :  $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

## 15. Experimental Procedure

### Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded, normalized, and ten sample images were displayed along with their class labels to verify correct loading.



Figure 3: Sample images from the CIFAR-10 dataset with their class labels.

### Task 2: Transfer Learning Model

Pre-trained ResNet50 architecture (with ImageNet weights and the top layer removed) was used as the convolutional base. This convolutional base was frozen, and three new layers were added on top: a Global Average Pooling layer, a Dense layer with ReLU activation (256 neurons), and another Dense output layer with Softmax activation (10 classes).

### Task 3: Model Training

The classifier head was trained for a maximum of 8 epochs with early stopping based on validation accuracy (patience=2), using the configuration below.

|                   |                                            |
|-------------------|--------------------------------------------|
| Optimizer         | Adam                                       |
| Learning Rate     | 0.001                                      |
| Batch Size        | 32                                         |
| Epochs            | 8 (Max)                                    |
| Early Stopping    | Patience = 2                               |
| Loss Function     | Categorical Cross-Entropy (one-hot labels) |
| Evaluation Metric | Accuracy                                   |

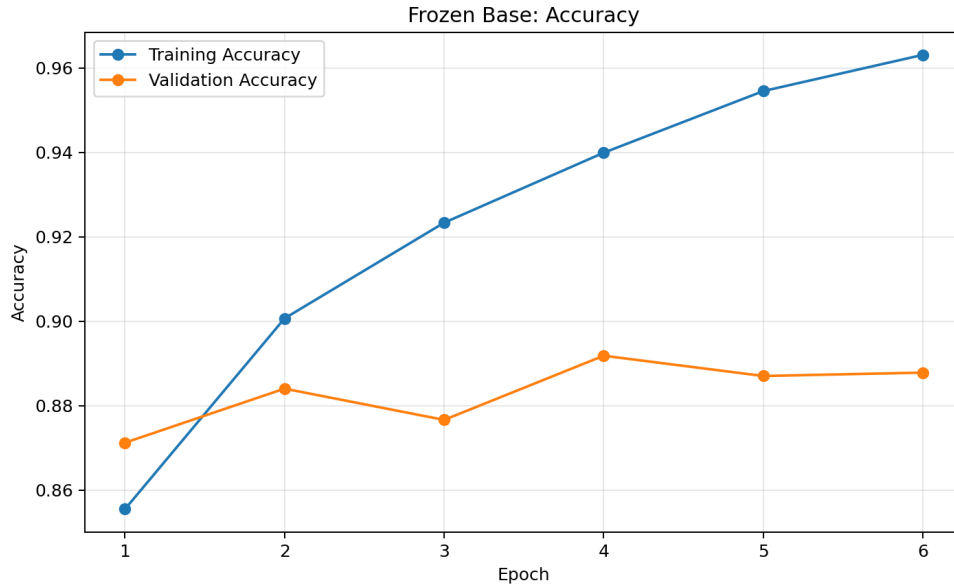


Figure 4: Training and validation accuracy over epochs (frozen base).

*Inference:* There was an increase in the training accuracy from 85.55% during epoch 1 to 96.31% during epoch 6, whereas the highest value of validation accuracy of 89.18% was reached in epoch 4 before early stopping. It is evident that the model showed great learning capacity with the frozen base.

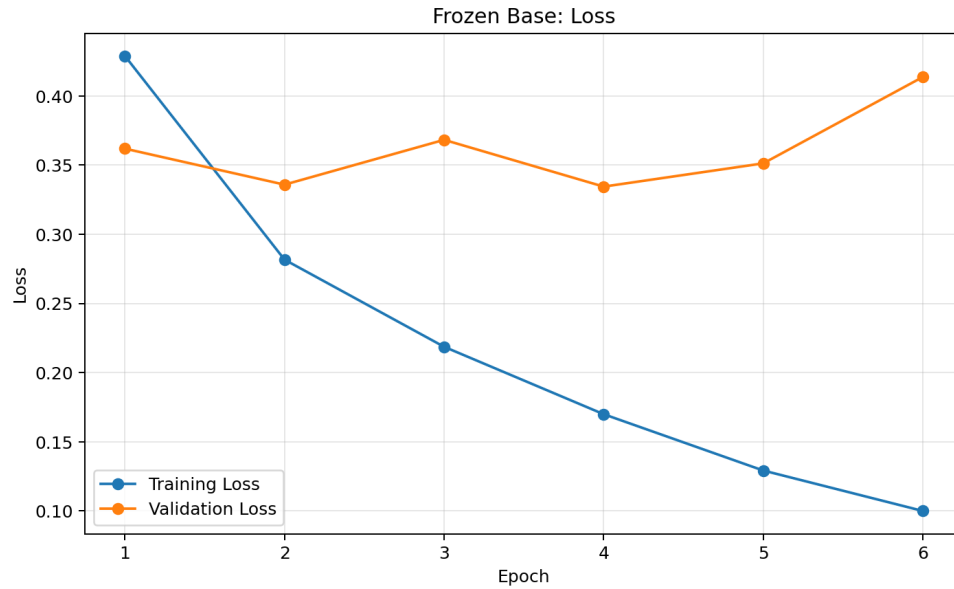


Figure 5: Training and validation loss over epochs (frozen base).

*Inference:* The loss on the training dataset went down from 0.4289 to 0.0999, while the loss on the validation dataset reached its minimum point of 0.3344 at epoch 4. The early stopping process loaded the weight for epoch 4 to ensure proper generalization.

#### Task 4: Fine Tuning

The last few convolutional layers of ResNet50 were unfrozen and the model was fine-tuned for 5 more epochs at a lower learning rate (0.0001) to let the pretrained features to adapt slightly to CIFAR-10.

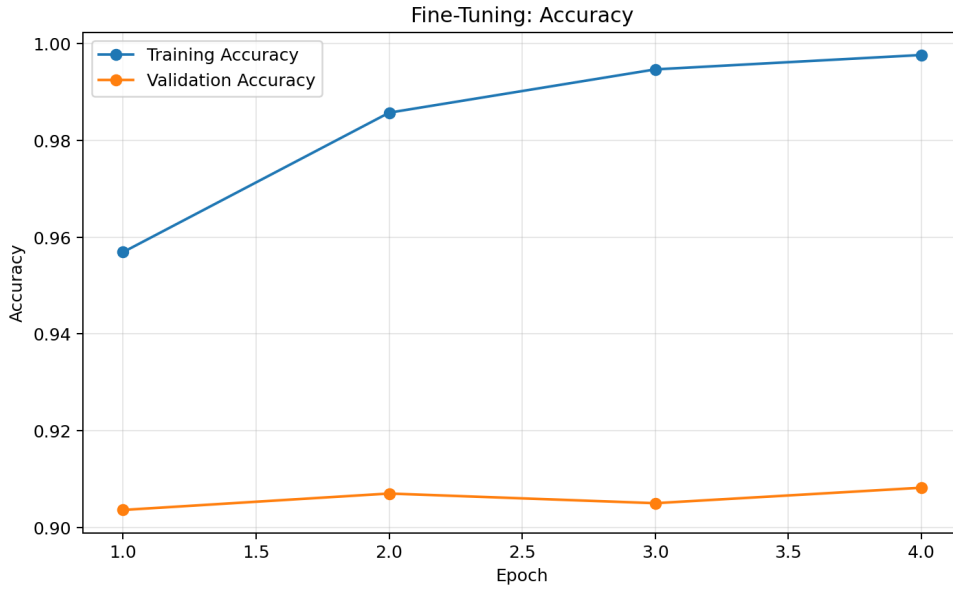


Figure 6: Training and validation accuracy after fine-tuning.

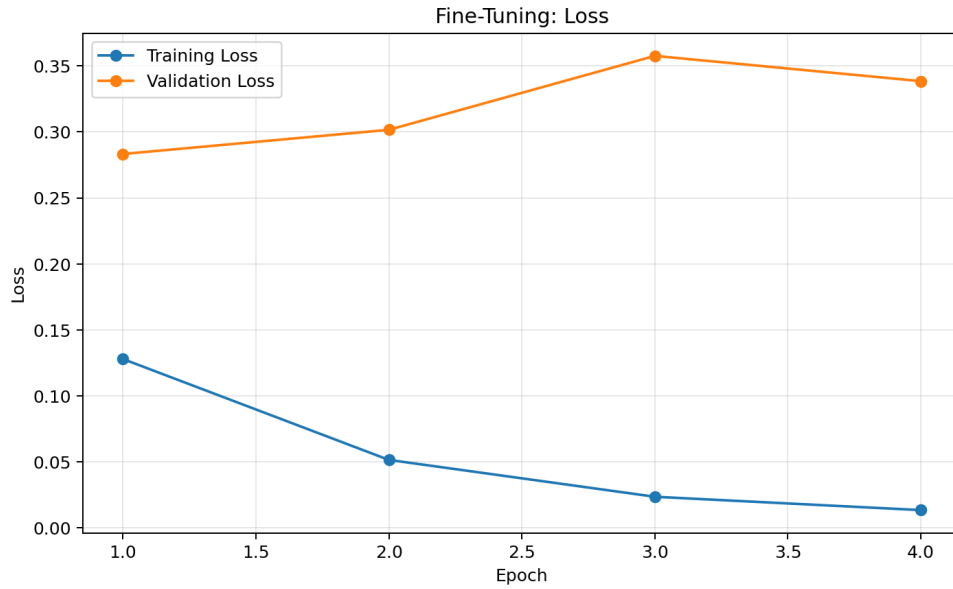


Figure 7: Training and validation loss after fine-tuning.

*Inference:* Fine-tuning resulted in great success as the model attained near-perfect training accuracy of 99.77% in epoch 4 while reducing the training loss to 0.0134. The validation accuracy increased as well to achieve a peak value of 90.82% in epoch 4 (note: final test accuracy achieved independently is mentioned in Section 18) while the validation loss was 0.3384. This shows that by unfreezing the last few layers and training with a very low learning rate, the model could learn the ImageNet features for the CIFAR-10 dataset without catastrophic forgetting.

## Task 5: Model Evaluation

The fine-tuned model was evaluated on the CIFAR-10 test set using accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.

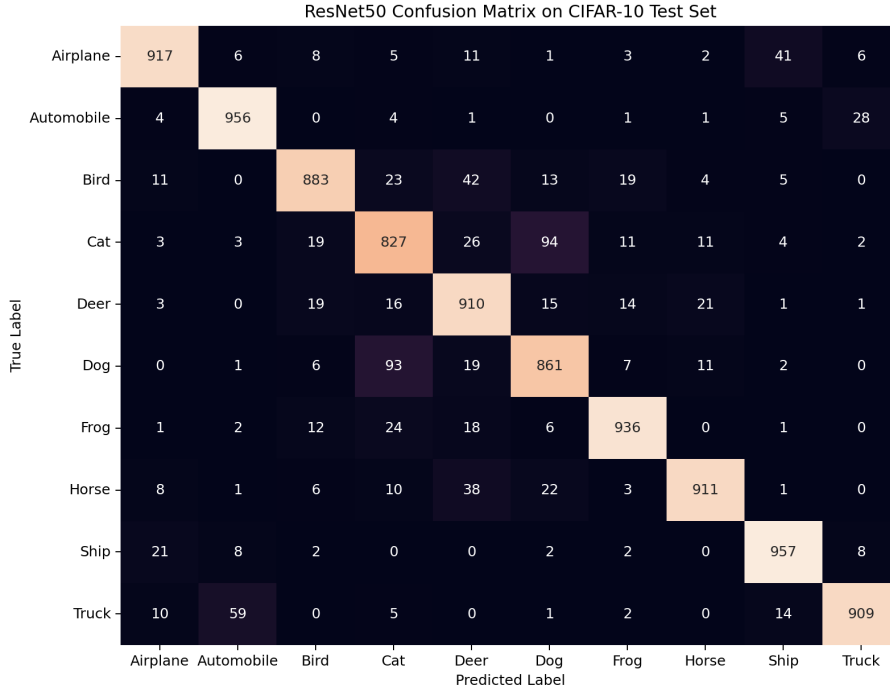


Figure 8: Confusion matrix of the fine-tuned ResNet50 model on the CIFAR-10 test set.

Table 1: Class-wise precision, recall, and F1-score on the CIFAR-10 test set

| Class      | Precision | Recall | F1-score |
|------------|-----------|--------|----------|
| Airplane   | 0.94      | 0.92   | 0.93     |
| Automobile | 0.92      | 0.96   | 0.94     |
| Bird       | 0.92      | 0.88   | 0.90     |
| Cat        | 0.82      | 0.83   | 0.82     |
| Deer       | 0.85      | 0.91   | 0.88     |
| Dog        | 0.85      | 0.86   | 0.85     |
| Frog       | 0.94      | 0.94   | 0.94     |
| Horse      | 0.95      | 0.91   | 0.93     |
| Ship       | 0.93      | 0.96   | 0.94     |
| Truck      | 0.95      | 0.91   | 0.93     |

The network demonstrated exceptional balanced and consistent results for all classes, with the macro-average precision, recall, and F1 scores close to 0.91. Classes that have unique structural characteristics, such as Automobile, Frog, and Ship, showed the best F1 scores (0.94). Animal classes that are visually similar to each other, for example, Cat (0.82) and Dog (0.85), had somewhat lower results due to the expected confusion between classes.



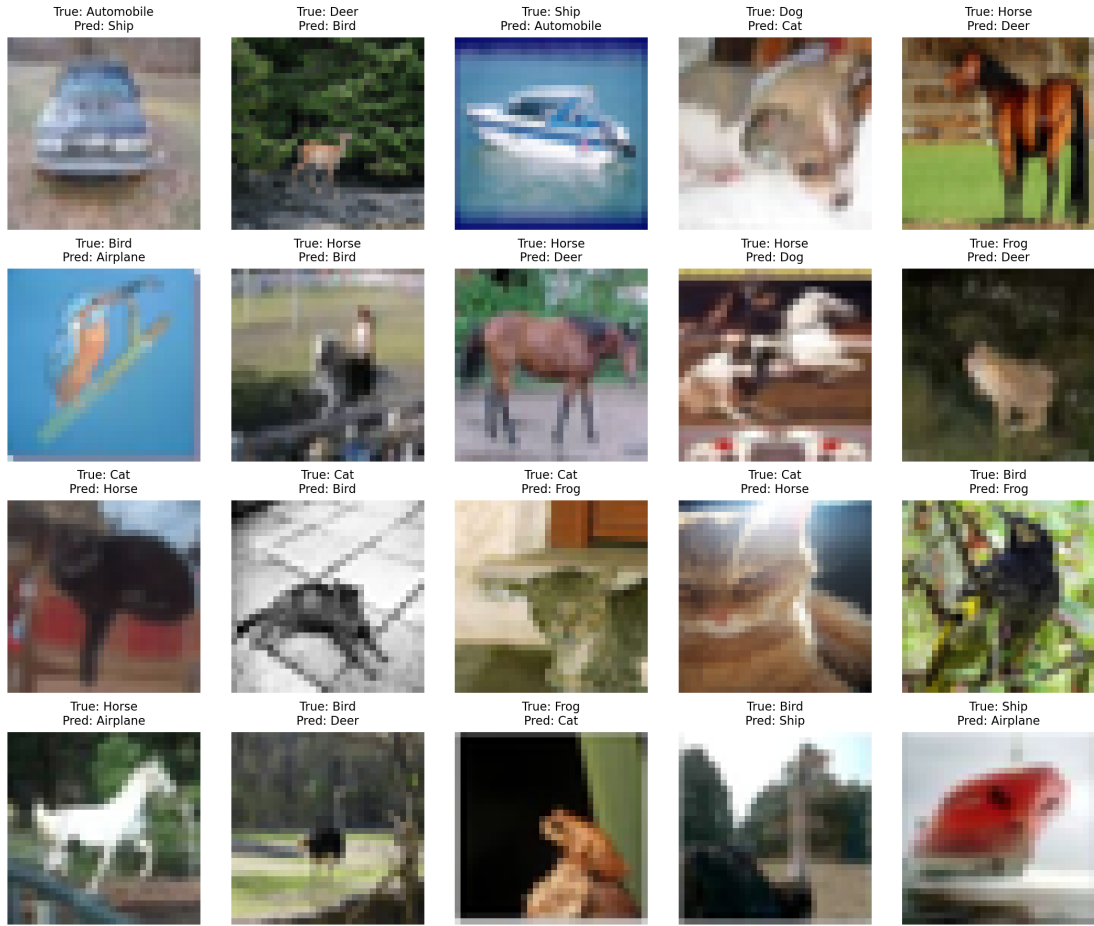


Figure 9: Sample misclassified test images with true and predicted labels.

## 16. Hyperparameter Study

Performance was compared across different hyperparameter settings, as summarized below.

Table 2: Hyperparameter study settings and observations (2-epoch quick comparisons)

| Hyperparameter | Values              | Observation                                                                                   |
|----------------|---------------------|-----------------------------------------------------------------------------------------------|
| Learning Rate  | 0.001, 0.0001       | 0.0001 performed slightly better (88.08% vs. 87.96%).                                         |
| Batch Size     | 32, 64              | Batch size 32 performed slightly better (88.62% vs. 88.08%).                                  |
| Optimizer      | Adam, SGD           | Adam slightly outperformed SGD (88.20% vs. 87.86%).                                           |
| Dense Units    | 128, 256            | 128 dense units marginally outperformed 256 units (88.76% vs. 88.28%).                        |
| Frozen Layers  | All Frozen, Partial | Partial unfreezing (fine-tuning) noticeably improved validation accuracy (90.24% vs. 88.74%). |

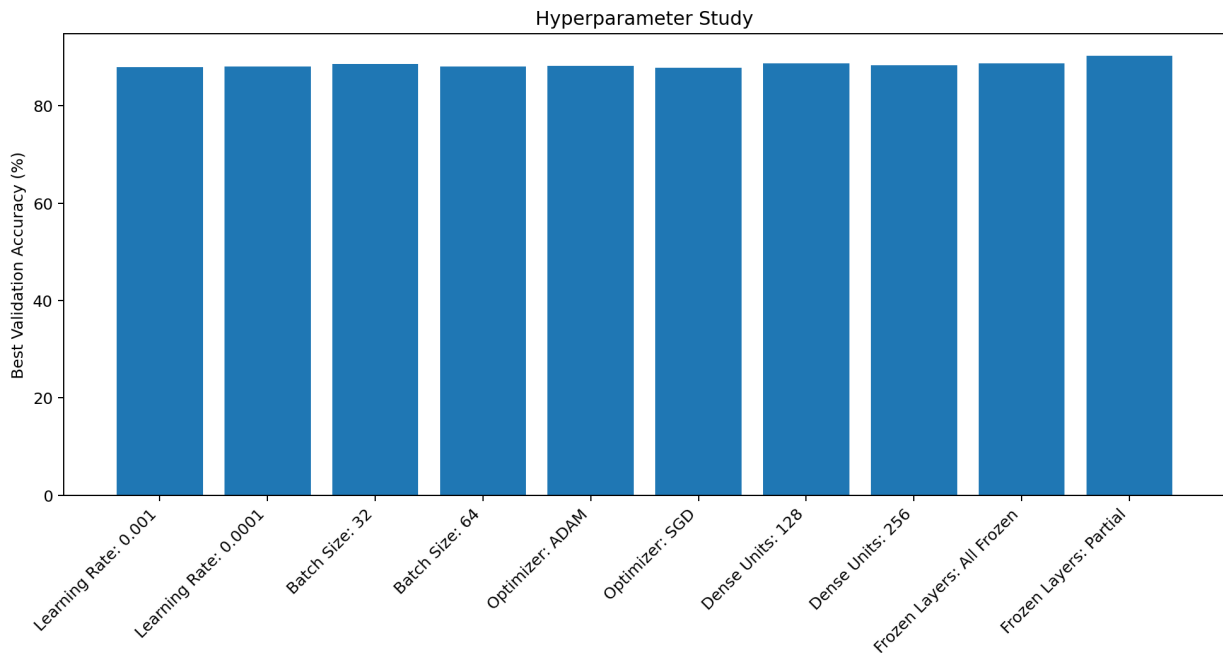


Figure 10: Validation accuracy across different hyperparameter settings (2-epoch quick comparisons).

*Note:* The hyperparameter study used short 2-epoch experiments and may not be directly comparable to the final fine-tuning experiment.

## 17. Results

### 17.1 Performance Metrics

| Metric            | Value                                                       |
|-------------------|-------------------------------------------------------------|
| Training Accuracy | 99.77                                                       |
| Testing Accuracy  | 90.67                                                       |
| Precision         | 0.91                                                        |
| Recall            | 0.91                                                        |
| F1-score          | 0.91                                                        |
| Training Time     | 8.33 min                                                    |
| Total Parameters  | 24,114,826 (4,986,634 trainable + 19,128,192 non-trainable) |

*Note on parameters:* The ImageNet ResNet50 model has about 25.6 million parameters. The number of parameters mentioned here is that of the modified ResNet50 model used in the experiment.

*Comment on accuracy:* High testing accuracy (90.67%) proves how well transfer learning can work despite resolution domain shift. Though ResNet50 was initially trained on  $224 \times 224$  ImageNet samples, this issue was easily solved in the experiment in the notebook via explicit resizing of  $32 \times 32$  CIFAR-10 samples to  $128 \times 128$  via `tf.image.resize` (antialias option). Additionally, the use of `keras.applications.resnet50.preprocess_input` function made the input pixel distribution match pre-trained weights. Along with proper tuning of the model, namely, a two-step strategy of training where initially a new custom classification head (Global Average Pooling + 256 neuron Dense layer) is trained for 8 epochs with the base frozen, and then the last 10 layers of the ResNet50 base are unfrozen and fine-tuned for 4 epochs with learning rate of  $10^{-4}$ , it made the model learn well.

## 17.2 Comparison of CNN Architectures

| Model     | Parameters     | Accuracy (%) | Training Time |
|-----------|----------------|--------------|---------------|
| LeNet-5   | $\approx 60K$  | Not measured | Not measured  |
| AlexNet   | $\approx 61M$  | Not measured | Not measured  |
| VGG16     | $\approx 138M$ | Not measured | Not measured  |
| GoogLeNet | $\approx 6.8M$ | Not measured | Not measured  |
| ResNet50  | 24.1M          | 90.67        | 8.33 min      |

## 18. Source Code

[GitHub Repository](#)

## 19. Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?

*Answer:* AlexNet proved that deep convolutional neural networks trained using GPUs can achieve unparalleled accuracy on large-scale datasets like ImageNet. This was an important step in the development of modern approaches including ReLU activation function, Dropout, and data augmentation, which initiated the deep learning revolution.

2. Why does VGG16 use only  $3 \times 3$  convolution filters?

*Answer:* Stacking of several layers using  $3 \times 3$  filters provides the same receptive field that would be provided by large-sized filters (for example, two layers using  $3 \times 3$  filters are equivalent to one layer using  $5 \times 5$  filters), but the parameters are minimized.

3. Explain the advantages of the Inception module.

*Answer:* In the Inception module, convolution operations with different filter sizes ( $1 \times 1$ ,  $3 \times 3$ , and  $5 \times 5$ ) and also max pooling are applied in parallel to extract information from the input data at multiple scales. But the use of  $1 \times 1$  convolutions in parallel

4. What is the purpose of residual learning?

*Answer:* The idea of residual learning uses skip connections by which the input and the output of a specific layer can be added together to ensure that the gradient passes straightly through the network. It is significant while training deep models because the problem of vanishing gradient is solved.

5. Differentiate LeNet and ResNet.

*Answer:* LeNet is a simple and shallow CNN consisting of 7 layers and having nearly 60,000 parameters, which can detect simple grayscale digits (such as MNIST and OCR). In contrast, ResNet is an advanced CNN architecture that has a very large number of parameters and uses skip connections to extract higher-level features from complicated and high-resolution color images.

6. What is Transfer Learning?

*Answer:* Transfer learning is the method by which one can fine-tune a model that has been trained on a dataset (e.g., ImageNet) for computer vision tasks on another dataset (e.g., CIFAR-10) using the knowledge acquired from the pretrained model.

7. Why is fine-tuning required?

*Solution:* In general, the weight of lower layers learns the generic features such as edges, textures, etc., which can be useful for most vision-related problems. On the other hand, the weight of higher

layers learns the features corresponding to the particular problem. In the fine-tuning phase, we generally update the weight of higher layers with small learning rates.

8. Explain the difference between dilated convolution and transpose convolution.

*Answer:* The dilated convolution enlarges the receptive field of the convolution process using spacing between filter values. Transpose convolution on the other hand can be referred to as learned upsampling process.

9. Why do pretrained models converge faster?

*Answer:* The pretrained network already has optimal weights for recognizing visual features. As we need to train the network for a few epochs to learn the new features, the time taken to converge is minimal.

10. Compare the computational complexity of LeNet and ResNet.

*Answer:* The complexity of the LeNet architecture is quite low and it could be trained using a CPU. Conversely, the complexity of ResNet architecture is quite high and training such an architecture requires a GPU.

## 20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

## 21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>